

Propuesta de Tesina

Postulante: Antonio Locascio

Director: Mauro Jaskelioff

Codirector: Exequiel Rivas

1. Situación del postulante

Al día 3 de mayo de 2019 tengo aprobadas 28 materias de la carrera de Licenciatura en Ciencias de la Computación. Actualmente estoy cursando Taller de Tesina y Seminario de Pasantía, y el cuatrimestre próximo cursaré Seguridad Informática y una optativa. Además, recientemente rendí el examen final de Ingeniería del Software 2, el cual todavía no ha sido corregido. Junto a la tesina, estas materias son lo último que me falta para finalizar la carrera.

Al no estar realizando ninguna actividad adicional, puedo trabajar en esta tesina con una dedicación de tiempo completo.

2. Título

Sistema de tipos bidireccional para un lenguaje con efectos algebraicos.

3. Motivación y objetivo general

El modelado formal de efectos computacionales, tales como entrada/salida o estado, permite razonar sobre estos efectos, mantenerlos controlados y obtener implementaciones eficientes. Uno de estos modelos es el de *efectos algebraicos*, que se basa en la idea de que el comportamiento impuro surge de ciertas operaciones que pueden ser tratadas por separado del resto del programa.

Para conocer y limitar los efectos que pueden ser ocasionados por un programa es necesario contar con un sistema de tipos con efectos (o sistema de efectos). En estos, a cada término de un dado lenguaje se le da un tipo que contiene información sobre

los efectos que puede invocar. Una dificultad del diseño de estos sistemas radica en que su simplicidad teórica, deseable para razonar y poder adaptarlo a lenguajes más complejos, usualmente se pierde en sus implementaciones.

En consecuencia, el objetivo general de esta tesina es formular e implementar un sistema de efectos para un cálculo simple con efectos algebraicos, que sirva como base para diseñar y experimentar con extensiones al lenguaje base.

4. Fundamentos y estado del conocimiento sobre el tema

4.1. Efectos algebraicos

Moggi [7] propuso modelar efectos computacionales encapsulándolos en mónadas y agregando ciertas operaciones que dan origen a estos efectos. Sin embargo, este enfoque contaba sólo con una semántica denotacional, y no estaba claro cómo darle una semántica operacional. Con el objetivo de dar semántica operacional a este enfoque, Plotkin y Power [10] proponen en cambio representarlos mediante un conjunto de operaciones que son fuente de efectos, junto a una teoría ecuacional que describe sus propiedades. Estos mismos autores exponen la relación teórica entre ambos abordajes, mostrando que las teorías algebraicas dan origen a mónadas [11].

Ciertas operaciones habituales, como manejar una excepción, no son operaciones algebraicas. Plotkin y Pretnar [9] muestran que el manejo de excepciones está dado por la interpretación de la operación que lanza la excepción en un modelo. Generalizando esta idea, se define la idea de *handlers* de operaciones para darle semántica a las operaciones de una teoría de efectos algebraicos.

Existe un número de lenguajes experimentales que implementan estas ideas de distintas formas. *Eff* [2, 1, 12] de Bauer y Pretnar, y *Koka* [5] de Leijen incorporan handlers similares a los de la teoría y cuentan con sistemas de tipos de efectos que proveen inferencia de Hindley-Milner. *Frank* [6] generaliza las funciones y los handlers por medio de *operadores* que pueden interpretar múltiples operaciones a la vez, convirtiéndose en *multihandlers*. Para este lenguaje se presenta un sistema de tipado de efectos bidireccional con polimorfismo de efectos. *Multicore Ocaml* [4] es una extensión de Ocaml con soporte para efectos algebraicos y handlers, pero sin tipado de efectos. Todos estos lenguajes omiten las ecuaciones, trabajando sobre modelos libres en teorías sin ecuaciones.

El área de efectos algebraicos es reciente y tiene muchos temas de investigación

actualmente activos. Algunos de ellos son la implementación y eficiencia de este tipo de lenguajes, tratamiento de multihandlers y la incorporación de las ecuaciones en la práctica.

4.2. Sistemas de tipos bidireccionales

No todos los sistemas de tipos pueden ser implementados algorítmicamente de manera directa, ya que muchos no están descriptos de una manera dirigida por sintaxis. Se dice que un sistema de tipos es dirigido por sintaxis si cada juicio de tipado tiene una derivación única, determinada por completo por la estructura sintáctica del término en cuestión [3].

Pierce y Turner [8] introducen una técnica llamada *tipado bidireccional* que permite hacer que las reglas de tipado sean dirigidas por sintaxis. Además de proveer un algoritmo de chequeado de tipos, esta técnica reduce el número de anotaciones de tipo necesarias, requiriéndolas únicamente en el *top-level*.

La idea central de los sistemas de tipos bidireccionales consiste en reemplazar el juicio de tipado estándar con dos nuevos juicios: uno que chequea que una expresión sea de un tipo dado y otro que sintetiza el tipo de una expresión. Se busca propagar la información de tipo hacia las subexpresiones alternando entre ambos juicios.

5. Objetivos específicos

En concreto, los objetivos de esta tesina son:

- Definir un cálculo simple con efectos algebraicos y handlers, separando las expresiones puras de las computaciones impuras.
- Diseñar un sistema de tipos con efectos bidireccional para este lenguaje, teniendo en cuenta la posibilidad de incorporar extensiones.
- Realizar una implementación del sistema anterior.

6. Metodología y plan de trabajo

El plan de trabajo sigue los objetivos específicos de la sección anterior. En primer lugar se definirá un lenguaje basado en *Core Eff* [1]. El segundo objetivo, el diseño

del sistema de efectos, se divide en dos tareas: la definición de las reglas de tipado y la prueba de seguridad del sistema. La última tarea corresponde a la implementación.

El sistema de tipos de *Frank* [6] sirve como evidencia que un tipado bidireccional para lenguaje con efectos algebraicos es viable. Sin embargo el lenguaje y sistemas de tipos de Frank es complejo y bastante lejano de la teoría de efectos algebraicos. En cambio, la idea principal de la tesina es obtener un lenguaje simple con tipado bidireccional cercano a la teoría.

6.1. Cronograma de trabajo

El cronograma tentativo de la tesina es el siguiente:

- Relevamiento del estado del arte y material bibliográfico: un mes.
- Diseño del lenguaje: un mes.
- Testeo y prueba de seguridad del lenguaje: un mes.
- Implementación: un mes.
- Escritura del informe: un mes.

Referencias

- [1] Andrej Bauer and Matija Pretnar. An effect system for algebraic effects and handlers. *Logical Methods in Computer Science*, 10(4), 2014.
- [2] Andrej Bauer and Matija Pretnar. Programming with algebraic effects and handlers. *J. Log. Algebr. Meth. Program.*, 84(1):108–123, 2015.
- [3] D. R. Christiansen. Bidirectional typing rules: A tutorial. <http://davidchristiansen.dk/tutorials/bidirectional.pdf>, 2013.
- [4] Stephen Dolan, Leo White, K. C. Sivaramakrishnan, Jeremy Yallop, and Anil Madhavapeddy. Effective concurrency through algebraic effects. In *Ocaml Workshop*, 2015.
- [5] Daan Leijen. Koka: Programming with row polymorphic effect types. In *Mathematically Structured Functional Programming 2014*. EPTCS, March 2014.

- [6] Sam Lindley, Conor McBride, and Craig McLaughlin. Do be do be do. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, POPL 2017, pages 500–514, New York, NY, USA, 2017. ACM.
- [7] Eugenio Moggi. Notions of computation and monads. *Inf. Comput.*, 93(1):55–92, July 1991.
- [8] Benjamin C. Pierce and David N. Turner. Local type inference. *ACM Trans. Program. Lang. Syst.*, 22(1):1–44, January 2000.
- [9] Gordon Plotkin and Matija Pretnar. Handlers of algebraic effects. In *Proceedings of the 18th European Symposium on Programming Languages and Systems: Held As Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2009, ESOP '09*, pages 80–94, Berlin, Heidelberg, 2009. Springer-Verlag.
- [10] Gordon D. Plotkin and John Power. Adequacy for algebraic effects. In *Proceedings of the 4th International Conference on Foundations of Software Science and Computation Structures*, FoSSaCS '01, pages 1–24, Berlin, Heidelberg, 2001. Springer-Verlag.
- [11] Gordon D. Plotkin and John Power. Notions of computation determine monads. In *Proceedings of the 5th International Conference on Foundations of Software Science and Computation Structures*, FoSSaCS '02, pages 342–356, London, UK, UK, 2002. Springer-Verlag.
- [12] Matija Pretnar. An introduction to algebraic effects and handlers. invited tutorial paper. *Electron. Notes Theor. Comput. Sci.*, 319(C):19–35, December 2015.